Seminar course

Formally Verified Systems

(aka “formal-seminar”)

Preliminary meeting

https://dse.in.tum.de/

Dr. Dennis Sprokholt



About me

Systems Research Group

https://dse.in.tum.de/team/



Dr. Dennis Sprokholt

Humboldt Research Fellow

About me



Communication:

Join us with TUM email address (@tum.de)
ls1-courses-tum.slack.com
#ws26-formal-seminar

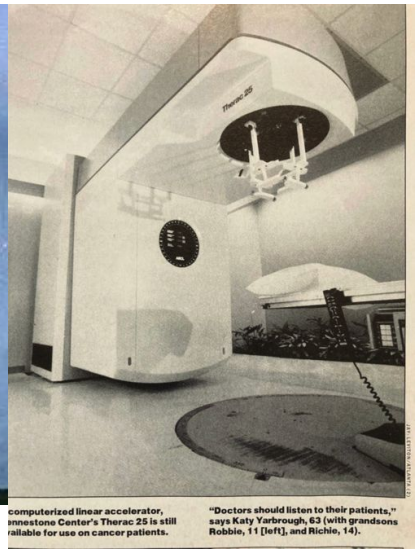
Context

The Need for Correct Software

Ariane flight V88



Therac-25

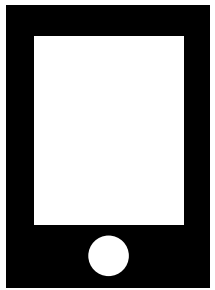


computerized linear accelerator, ennestone Center's Therac 25 is still vailable for use on cancer patients.

"Doctors should listen to their patients," says Katy Yarbrough, 63 (with grandsons Robbie, 11 [left], and Richie, 14).

Everyday software





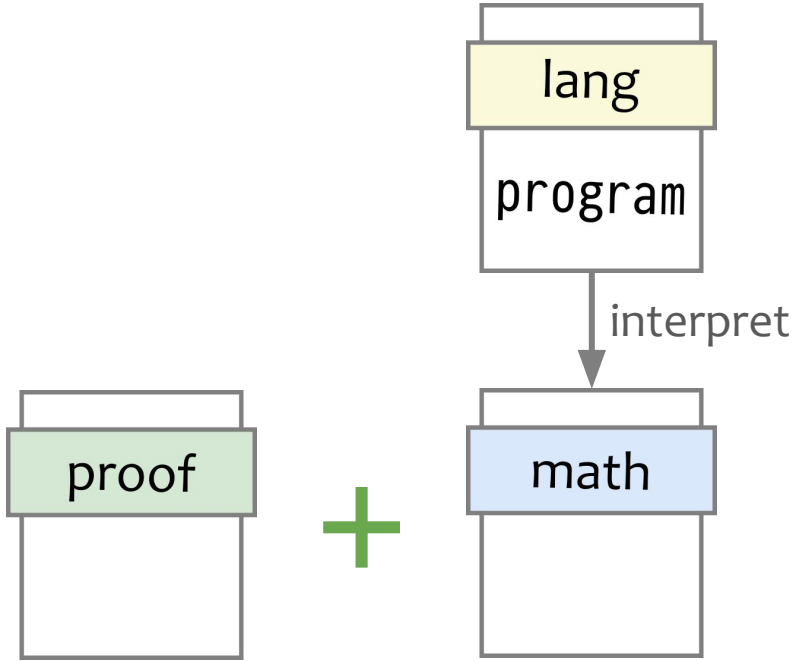
Software must be reliable – both *safety-critical* and *everyday* software

Writing Correct Programs is Hard

- Large state spaces
- Concurrency
- Hardware varies
- Environment varies

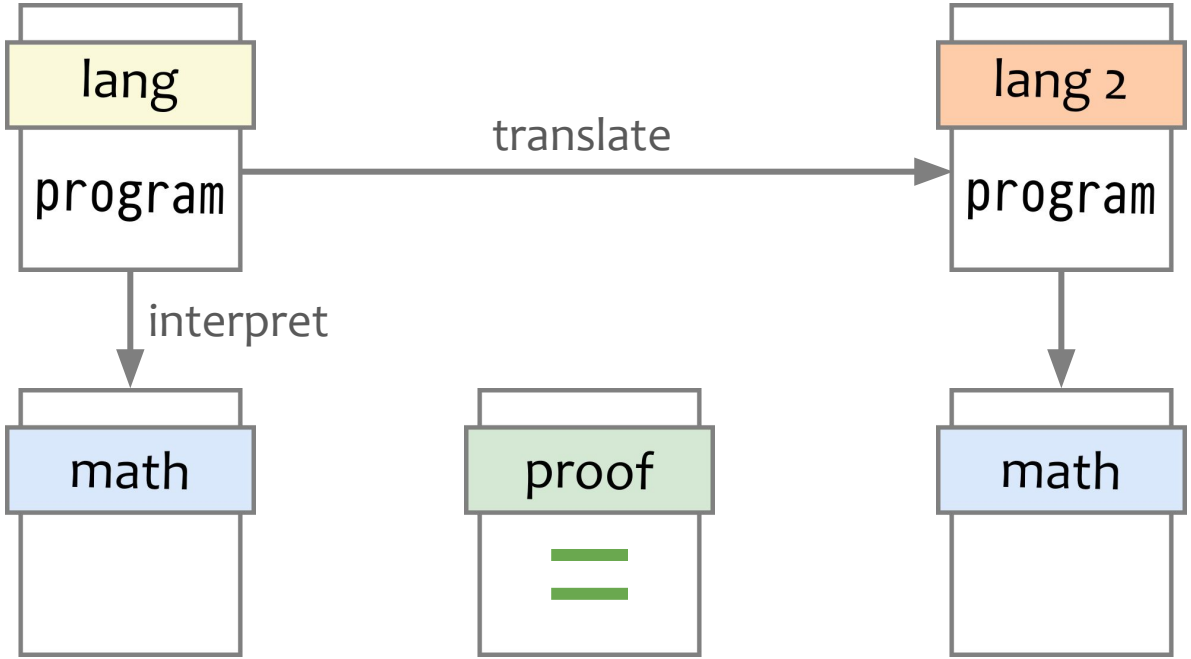
Correctness by Formal Proof

- Full correctness
- Properties
  - Data race freedom
  - No overflows
  - Memory safety



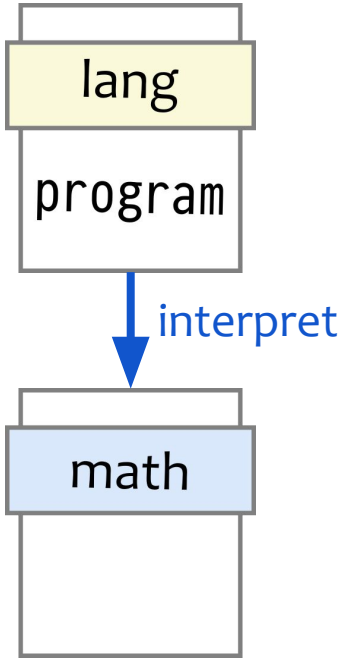
Correctness by Formal Proof

- Compilation
- Optimization



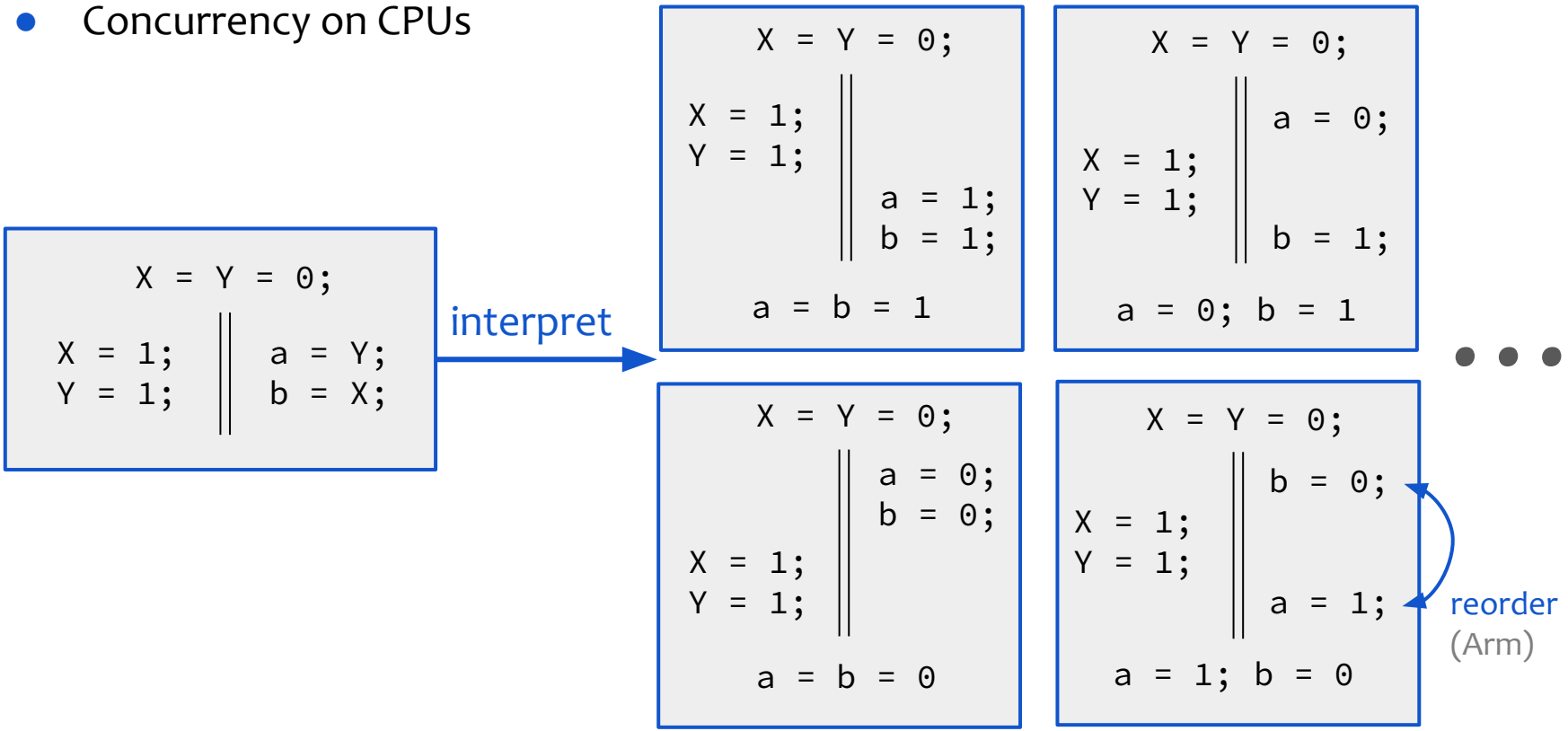
Semantics

- Assign *meaning* to programs, by an interpretation function
- Relevant semantics vary by domain
  - Weak memory semantics
  - ISA semantics



Weak Memory Semantics

- Concurrency on CPUs



Machine Readable Architecture Specifications

- Describes how an ISA behaves
  - How instructions affect registers and memory

SAIL[1]

```
function clause execute (ITYPE (imm, rs1, rd, op)) = {
  let rs1_val = X(rs1); // read source register
  let immext : xlenbits = EXTS(imm);  // sign-extend
  let result : xlenbits = match op {
    RISCV_ADDI => rs1_val + immext,
    ...
  };
  X(rd) = result; // write target register
  true
}
```

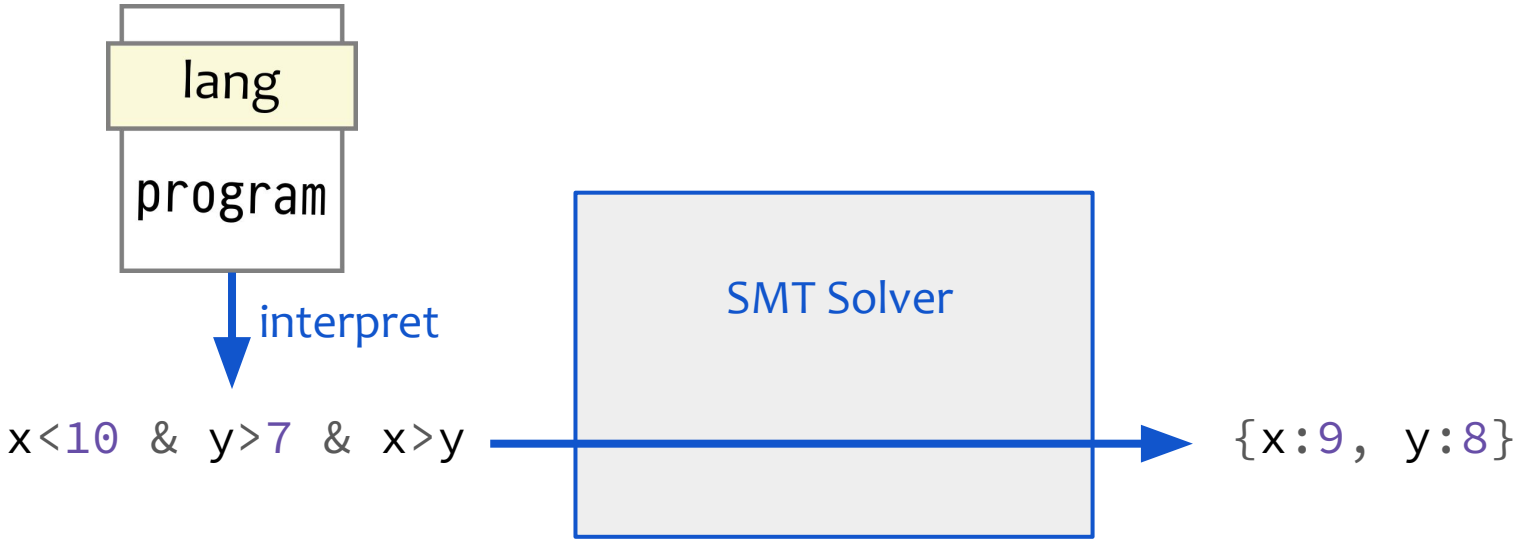
[1] https://github.com/rems-project/sail

Verification Techniques

- Verification techniques vary in the provided guarantees
  - (Unit) Testing
  - Fuzzing
  - SAT/SMT-based approaches
  - Mechanized proofs
- Vary also in developer effort

Lightweight Verification

- SMT solvers check (un)satisfiability of logic formulae
  - E.g., for Translation validation of LLVM[1]



[1] Alive2: Bounded Translation Validation for LLVM

Mechanized Proofs

- A programming language in which to write proofs
  - Proof assistant helps write proofs

Agda[1]

```
+-assoc-0 : ∀ (n p : ℕ)
  → (0 + n) + p ≡ 0 + (n + p)
+-assoc-0 n p =
  begin
    (0 + n) + p
  ≡⟨⟩
    n + p
  ≡⟨⟩
    0 + (n + p)
  ∎
```







[1] Programming Language Foundations in Agda. https://plfa.github.io/Induction/

Format

Tentative topics

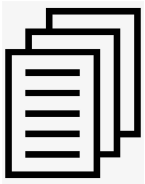
Papers from top conferences in systems and programming languages:
PLDI, ASPLOS, OSDI, SOSP, EuroSys

Tentative topics
Interactive theorem proving
Correct-by-construction systems
SMT-based verification
Hardware architecture specification and verification
Weak memory semantics
Machine-readable architecture specifications
Verified compilation
Correctness across the hardware/software interface
...

Bird’s eyes view



Team
(2 students per team)



Research papers
(Top conferences in systems and programming languages)





Understand



Research ideas

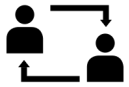




1 presentation



1 short report



Peer-reviewing

Overview

Phase I

Kick-off



Phase II: Understand & explore

Understand



Presentation

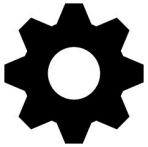


Phase III: Research

Design



Implement (Bonus)



Phase IV: Report & review

Report



Peer-review





Phase I: Kick-off meeting



Format and motivation
(all participants meeting)



Team formation
(2 students per team)



Paper selection
(Top conferences in systems and programming languages)

The first week

NOTE

1. A list of papers will be provided for FCFS bidding
2. Paper presentation guidelines will be provided for the next phase

Phase II: Understand & explore



Understand the paper(s)

Focus

1. **Understand** the paper and related work
2. Also **explore** a "laundry list" of research ideas/directions





Paper presentation

Focus

1. Explain the work/related work (**"why?"** and **"how?"**)
2. Explain and discuss all possible research directions
3. Pick a research direction

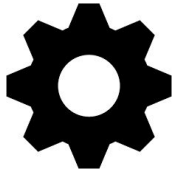


Phase III: Research



Research work

Focus:
Indepth research work to nail-down the problem and detailed approach to solve it!



Research prototype

Bonus:
(Optional)
"Build the system to solve it!" and show us the working idea and associated results



Phase IV: Report & review



Report

Focus

Prepare a single “short & sweet” report summarizing

(a) Paper

(b) Research work



Peer-review

Focus

Give constructive (positive and critical) feedback for

(a) Paper summary

(b) Research work

END.

Overall timeline

Phase I

Kick-off



Phase II: Understand & explore

Understand



Presentation

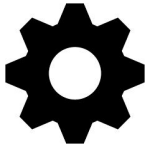


Phase III: Research

Design



Implement (optional)

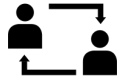


Phase IV: Report & review

Report



Peer-review



Milestone #1: Team formation & paper selection

Milestone #2: Paper presentations

Milestone #3: Research work complete

Milestone #4: Report submission

Milestone #5: Peer-reviewing

1 week

3 weeks

2 weeks

3 weeks

1 week

2 weeks

Meeting

Meeting

Organization

- Format
  - Team-based seminar course (2 students per team)
- Communication
  - Slack for announcements and information sharing
  - Hotcrp for report submission and peer-reviewing
- Meetings (**in-person, attendance is compulsory**)
  - **Meeting #1:** Kick-off
  - **Meeting #2:** Paper presentation

Learning goals

- Learn about the cutting-edge research in computer systems
- Promote critical thinking
- Cultivate an environment for innovation
  - To push the boundaries by advancing the state-of-the-art
- Improve scientific skills
  - Presentation
  - Writing
  - Communication: discussion and arguing
  - Mentorship: giving feedback and moderating discussion
- Encourage system building and evaluation
  - Learn by building, breaking, and benchmarking systems
- Importantly, to have fun!

Code of conduct

- University plagiarism policy
  - https://www.cit.tum.de/fileadmin/w00byx/cit/Studium/Im_Studium/Thesis/leitfaden-en_2016Jun22.pdf

- Decorum
  - Promote freedom of thoughts and open exchange of ideas
  - Cultivate dignity, understanding and mutual respect, and embrace diversity
  - Racism and bullying will not be tolerated

Contact

- Dr. Dennis Sprokholt
  - dennis.sprokholt@tum.de

- All seminar-related info: https://github.com/TUM-DSE/seminars



Communication:

Join us with TUM email address (@tum.de)